CS781 Capstone Project: In-House Legal Intelligence System

Ayush Mangal
240242
ayushn24@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

Legal documents often contain confidential information that law firms cannot freely share with external language processing services. This creates a need for legal NLP systems that can be deployed and operated locally. In this project, I developed a legal NLP pipeline covering seven tasks, with the Phase 1 foundation model restricted to the 50M–300M parameter range. First, I pretrain four compact transformer backbone families from scratch on a 200M-token domain corpus (Supreme Court 45%, High Courts 35%, Acts 15%, Tribunals 5%) using a custom 32,000-token BPE tokenizer. The 59.23M-parameter Hybrid Prefix-LM achieved top foundation pretraining accuracy (BERTScore F1 of 0.7965). I evaluated different ways of adapting the pretrained models to the remaining six tasks, including parameter-efficient fine-tuning and knowledge distillation. The experiments showed that the best-performing approach differed across tasks. Bidirectional attention and LORMA worked well for some classification tasks, while knowledge distillation improved the summarization results, reaching peak scores in NER (Strict Macro-F1 0.3735), Rhetorical Roles (0.3769), Court Judgment Prediction (0.5440), Statute Identification (0.1773), Summarization (0.8219), and Prior Case Retrieval (0.1542).

1 Competition Result

Codabench username: a_240242 **Roll number:** 240242

Table 1: Official phase scores and ranks (recorded November 9, 2026).

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1: 0.796455	141	4
2	Legal NER	Strict Macro-F1: 0.373512	109	7
3	Rhetorical role prediction	Macro-F1: 0.376948	127	7
4	Court judgment prediction	Macro-F1: 0.544039	127	7
5	Legal statute identification	Macro-F1: 0.177300	133	4
6	Abstractive summarization	BERTScore F1: 0.821925	117	5
7	Prior Case Retrieval (PCR)	Micro-F1@7: 0.154164	131	5

Total valid Codabench submissions across all phases: 39 **Date ranks were recorded:** November 9, 2026

Submission Compliance Statement: I fully satisfied the mandatory minimum requirement of 4 distinct valid submissions per phase. Every submission evaluated a unique architecture or PEFT configuration.
Repository: https://github.com/ayushyra222324/Legal-Intelligence-System

2 Introduction and Problem Description

Legal institutions deal with large collections of documents, such as court judgments, statutory acts, procedural filings, and tribunal orders. Processing these documents manually can be time-consuming, especially when information needs to be retrieved from multiple cases. Although commercial LLMs can process legal documents, their use in law firms raises concerns about client confidentiality, data privacy, API costs, and the reliability of generated responses. General-purpose models may also have difficulty handling Indian legal citations, statutory references, and lengthy court judgments [1].

The objective of this project was to develop a legal intelligence system using compact language models that could be trained and evaluated within the given computational and data constraints.The pipeline covers seven core operational phases:

- **Phase 1 (Legal LM Pretraining):** Autoregressive and hybrid pretraining of domain foundation models from scratch (50M–300M parameter bound) on Indian legal text; evaluated via BERTScore F1 [2] on 128-word continuations.
- **Phase 2 (Legal Named Entity Recognition):** Sequence tagging over 12 legal entity classes (e.g., Court, Statute, Judge, Provision); evaluated via Strict Macro-F1 with exact character offset matching.
- **Phase 3 (Rhetorical Role Prediction):** Sentence-level discourse classification into 13 structural categories (e.g., Facts, Argument, Ratio, Precedent); evaluated via Macro-F1.
- **Phase 4 (Court Judgment Prediction & Explanation):** Binary decision outcome classification (0/1) for long judicial rulings; evaluated via Macro-F1.
- **Phase 5 (Legal Statute Identification):** Multi-label prediction across 100 distinct statutory provisions from case facts; evaluated via Macro-F1 under extreme target label sparsity.
- **Phase 6 (Abstractive Summarization):** Sequence-to-sequence document summarization from 1536-token case texts; evaluated via BERTScore F1, ROUGE-1, and ROUGE-L [3].
- **Phase 7 (Prior Case Retrieval):** Dense bi-encoder retrieval returning the top 7 relevant legal precedents for a query case from a candidate pool; evaluated via Micro-F1@7.

Design Constraints: Phase 1 parameter bounds were strictly locked between 50M and 300M parameters. External pretrained weights (e.g., LLaMA, RoBERTa, T5) and outside legal corpora were prohibited. Compute was constrained to Colab/Kaggle dual NVIDIA Tesla T4 GPUs. Downstream evaluation enforced exact character offset boundary matches for NER, highly imbalanced statute counts, and BERTScore for summarization.

Research Questions: The experiments focused on four main questions: (1) Does hybrid prefix-attention pretraining outperform pure causal decoding for structural legal continuation? (2) How effectively do PEFT methods (LoRA [4], QLoRA [5], LORMA [6], Prefix Tuning [7], BitFit [8]) transfer compact representations across sequence tagging, classification, and retrieval? (3) Does

converting causal decoders to bidirectional self-attention during fine-tuning resolve long-context decay? (4) Can logit-level knowledge distillation [9] compress fine-tuned teachers into compact student architectures without degrading accuracy?

3 Data Analysis and Preprocessing

3.1 Sources and Official Splits

All data was drawn strictly from the official [12] repository. Pretraining data comprised 200M tokens split across Supreme Court rulings (45%), High Court decisions (35%), Statutory Acts (15%), and Tribunal orders (5%). Downstream phases utilized official evaluation splits: Phase 2 contained 35 test-fold documents; Phase 3 evaluated 10 full test documents (sentence-level); Phase 4 evaluated 1,517 test judgment cases; Phase 5 covered multi-label targets across 100 statute IDs; Phase 6 evaluated 1536-token documents paired with target summaries; and Phase 7 comprised dense query and candidate case pools.

3.2 Corpus Statistics that Informed Design

Exploratory data analysis revealed critical domain characteristics that directly guided our architectural design:

- **Phase 1 Source Shift:** Statutory Acts exhibited dense numerical sections and formal clauses (1.26 tokens/word), whereas High Court and Tribunal documents contained long narrative legal arguments and irregular formatting (1.41 tokens/word). Based on these differences, I used documents from all four sources during pretraining instead of relying on a single document type.
- **Phase 2 Entity Imbalance & Length:** Background "O" tokens accounted for 82.4% of all tokens across the 12 entity types. The entity spans also varied considerably in length. Some court names occupied only one token, while longer statutory titles extended to 14 tokens.
- **Phase 3 Role Skew:** Rhetorical roles like Facts and Arguments dominated court transcripts ($> 62\%$ of sentences), while critical roles like Dissent and Precedent Overruled appeared in less than 1.4% of samples. Since the less frequent roles were underrepresented, I used class weights during training to reduce the effect of this imbalance.
- **Phase 4 & 7 Document Length:** Full judgment texts averaged 4,200 words. In many judgments, the final part of the document contained the ruling or order, while the beginning contained the case details. This observation motivated the use of both the beginning and the end of the document as model input.
- **Phase 5 Multi-Label Sparsity:** Across 100 target statute IDs, each document activated an average of only 1.84 labels, leading to a 98.16% zero-label matrix. With a default threshold of 0.50, the model predicted very few positive labels, resulting in low recall.
- **Phase 6 Summary Compression & Source Reuse:** 7,030 training pairs (400 held-out validation) and 100 test documents. Source documents averaged 3.1k words (p90 7.9k), while target summaries averaged $\sim$640 words ($\sim$22% of source length). Only 23% of reference summaries fit within 480 tokens. Since most reference summaries were longer than the available generation context, I needed to select relevant parts of the source document before generating the summary.

Table 2: Corpus statistics across all seven evaluation phases including validation splits.

Phase	Task Split / Size (Train / Val / Test)	Avg Input Length	Target Vocab / Class Count	Domain Feature & Handling
P1	200M train / 5M val tokens	1024	32k BPE Vocabulary	Blend: SC 45%, HC 35%, Acts 15%, Trib 5%
P2	3 folds (fold1 train / fold2 val / fold3 test)	1024 / block	12 BIO Entity Types	Exact char offset map; BIO Viterbi decoding
P3	80 train / 10 val / 10 test docs (2.1k val sents)	512 / seq	13 Discourse Roles	Weighted CE; sentence BiLSTM context
P4	31.7k train / 994 val / 1,517 test cases	2048 dual	Binary Outcome (0 / 1)	Dual-anchor (1024 head + 1024 tail) packing
P5	42.1k train / 10.1k val / 13.0k test docs	1024	100 Statutes (Multi-label)	98.2% sparsity; tuned threshold ($\tau = 0.10$)
P6	7,030 train (400 val) / 100 test pairs	1536	Abstractive Summary	Nucleus sampling ($T = 0.7$, $p = 0.9$)
P7	827 train / 118 val / 237 test queries	1024	Dense PCR (Top-7)	Bi-encoder; InfoNCE loss ($\tau = 0.05$)

3.3 Preprocessing

Tokenization & Citation Preservation: A custom byte-level BPE tokenizer (vocab size 32,000) was trained from scratch using "tokenizers". The vocabulary explicitly preserved legal section symbols (§) and citation structures to prevent subword fragmenting of Indian legal citations (e.g., *AIR 1954 SC 282*).

Phase 2 Character Offset Alignment: One issue in Phase 2 was maintaining the original character positions after tokenization. I used the offset mapping returned by the tokenizer to map each subword token back to its corresponding position in the original text. Subtoken boundaries were aligned back to raw text indices, preventing silent zero scores caused by whitespace normalization.

Dual-Anchor Chunking: For long documents in Phases 4, 5, and 7, I implemented a *Dual-Anchor Chunking* strategy. Simply taking the first 2,048 tokens would have excluded the ending of many long judgments. To retain information from both parts, I selected the first 1,024 tokens and the last 1,024 tokens and combined them into a single input.

Phase 6 Extract-then-Abstract Context Selection: To process 3.1k-word legal judgments within a 1536-token context window for Phase 6 abstractive summarization, I first selected relevant sentences from the original judgment. The selection process considered TF-IDF scores, sentence positions, and the presence of relevant cue words. The selected sentences were then used as input to the decoder.

4 Approach and Model Evolution

4.1 Phase-1 Foundation Models

I designed and pretrained four compact transformer backbones from scratch to compare architectural suitability for legal language modeling:

1. **Grouped-Query Attention (GQA) Causal LM (∼63.91M params):** 8 decoder layers, $d_{\text{model}} = 768$, $d_{\text{ff}} = 2048$, 12 query heads grouped into 4 KV heads (3:1 GQA ratio), RMSNorm, and RoPE [10]. Reached final validation loss of 5.3007.

2. **Custom Legal Encoder-Decoder (T5-Style, ∼105.71M params):** 6 encoder layers and 6 decoder layers, $d_{\text{model}} = 768$, 12 heads, $d_{\text{ff}} = 2048$, with cross-attention [11]. Achieved lowest pretraining validation loss (3.9352).

3. **Hybrid Prefix-LM / Hybrid Encoder (∼59.23M params):** 8 layers, $d_{\text{model}} = 768$, 12 heads, pairing bidirectional self-attention over prompt prefixes with causal masking for target generation. Reached validation loss of 4.4334.

4. **Modern Decoder (LLaMA-Style, ∼134.11M params):** 12 causal decoder layers, $d_{\text{model}} = 768$, 12 heads, SwiGLU MLP activations, RMSNorm, and RoPE. Reached validation loss of 4.2547.

4.2 Transfer Protocol (Phases 2–7)

For Phases 2–7, I experimented with different fine-tuning methods using the available pretrained backbones. These included LORMA, QLoRA, Prefix Tuning, and knowledge distillation. The adaptation method and task-specific model components varied across phases.

- **Linear-Out Rank-Minimizing Adaptation (LORMA):** I used LORMA adapters with ranks ranging from 16 to 128. The adapters were applied to the attention and MLP projections, while the original backbone weights were kept frozen.

- **Bidirectional Attention Conversion:** I converted causal decoders (Modern Decoder & GQA) into bidirectional encoders by setting is_causal = False during sequence tagging (P2) and sentence classification (P3). This allowed each token to use information from both the preceding and following tokens during prediction.

- **Knowledge Distillation (KD):** I also experimented with knowledge distillation. In this approach, a smaller student model was trained using the output distributions of a larger fine-tuned teacher, along with the task's original training objective.

- **Prefix Tuning & Soft Prompts:** I Prepended 16–32 continuous learnable virtual key-value prefix vectors across layers, projected via reparameterization MLPs.

4.3 Phase 2 — Legal NER

For Phase 2, I used the 12-layer Modern Decoder as a bidirectional encoder by disabling its causal attention mask. I then applied rank-128 LORMA adapters and added a token classification head. Token representations were passed to a linear classification head penalizing non-entity background tokens via class-weighted cross-entropy. This setup achieved the top Strict Macro-F1 of 0.3735.

In contrast, standard Hugging Face PEFT LoRA on the Hybrid Encoder failed to structure entity boundary transitions (Strict F1: 0.0022), and sequence-to-sequence LORMA suffered token-classification optimization failure (Strict F1: 0.0000).

4.4 Phase 3 — Rhetorical Role Prediction

Models classified sentence sequences into 13 structural categories. The best result in Phase 3 came from the Modern Decoder with bidirectional attention and rank-128 LORMA adapters. I used mean pooling to obtain sentence representations, which were then used for classification. This configuration achieved a Macro-F1 of 0.3769. Causal GQA models restricted to last-token pooling scored significantly lower ($\sim$0.264 to 0.271 F1). Distilling the Modern Decoder teacher into a 3-layer TinyStudent (10.56M params) retained strong accuracy (0.2953 F1), outperforming all frozen GQA variants.

4.5 Phase 4 — Court Judgment Prediction

Binary decision outcomes were predicted using Dual-Anchor 2048-token inputs. Modern Decoder + LORMA ($r = 16, \alpha = 32$) achieved the top Macro-F1 of 0.5440. I also trained a smaller two-layer GQA student using knowledge distillation. It achieved a Macro-F1 of 0.5307 and a recall of 0.7677, which was close to the teacher model's performance. BitFit (bias-only tuning) fine-tuned just 53K parameters while maintaining competitive performance (0.4691 F1).

4.6 Phase 5 — Legal Statute Identification

LSI required multi-label prediction across 100 statutory acts. Binary Cross-Entropy (BCE) loss over sparse targets ($\sim$98% zeros) pushed raw output logits deep into negative territory. I observed that the model produced very low logits for most statute labels. I therefore experimented with lower prediction thresholds instead of using the default value of 0.50. Lowering the decision threshold from $\tau = 0.50$ to $\tau = 0.10$ enabled GQA + LORMA ($r = 16$) to reach the peak Macro-F1 of 0.1773. Soft Prompt Tuning (16 tokens) on the Encoder-Decoder backbone achieved 0.1007 Macro-F1 with only 89K trainable parameters.

4.7 Phase 6 — Abstractive Summarization

Using 1536-token case documents, models generated concise legal summaries using nucleus sampling ($T = 0.7, p = 0.9$). Among the tested configurations, the three-layer Modern Decoder student achieved the highest BERTScore F1 of 0.8219. Interestingly, its score was higher than that of the six-layer teacher model, which achieved 0.8148. The Encoder-Decoder model with QLoRA ($r = 16$) achieved 0.7826 BERTScore F1 with strong n-gram overlap (ROUGE-1: 0.2927). Conversely, causal decoder-only prompting on the Hybrid Encoder suffered n-gram collapse (ROUGE-1: 0.0031).

4.8 Phase 7 — Prior Case Retrieval (PCR)

PCR was framed as dense bi-encoder metric learning over 1024-token texts under an in-batch InfoNCE contrastive loss ($\tau = 0.05$) on L_2-normalized embeddings. Modern Decoder + LORMA achieved the top Micro-F1@7 of 0.1542, followed closely by Encoder-Decoder QLoRA (Micro-F1@7: 0.1453). Prefix Tuning performed poorly in this phase, achieving a Micro-F1@7 of only 0.0118. This suggested that the virtual prefix parameters were not sufficient to adapt the frozen encoder for the retrieval task.

5 Experiments and Implementation Details

5.1 Compute and Training Protocol

All pretraining and fine-tuning runs were executed on dual NVIDIA Tesla T4 GPUs (16GB VRAM per GPU). I used PyTorch's automatic mixed precision with FP16 to reduce memory usage during training. AdamW was used as the optimizer, with linear warmup followed by cosine learning-rate decay. The Phase 1 experiments took approximately 5.5–10.4 hours per run, while downstream experiments generally took between 0.5 and 1.9 hours. I adjusted the learning-rate schedule according to the available training time. To avoid losing training progress when a job was interrupted, I saved Phase 1 checkpoints every 20 minutes and saved checkpoints during downstream evaluation. The training scripts also supported resuming from previously saved checkpoints.

Table 3: Hyperparameters of the final model in each phase.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	59.23	114.27	114.27	114.27	88.21	70.39	68.45
Context length	1280	1024	512	2048	2048	1536	1024
Peak LR	3e-4	1e-3	1e-3	2e-4	3e-4	5e-4	2e-4
Batch × accum.	2×4	2×8	8×4	4×4	4×4	2×1	8×1
Epochs / steps	1 / 9.7k	25 / 500	10 / 2.6k	3 / 78	3 / 150	3 / 1.2k	3 / 900
Loss Objective	Causal CE	W-CE	CE	CE	BCE	KD	InfoNCE
Decoding / threshold	Greedy	Exact	Mean	Argmax	$\tau = 0.10$	Nucleus	L2 Sim

5.2 Local Evaluation

For local evaluation, I held out 10–20% of the available training data for validation. I used stratified sampling where applicable. The local scores were generally close to the corresponding CodaBench results. However, in Phase 2 (L-NER), local unweighted token-F1 diverged significantly from official Strict Macro-F1 until strict character offset matching and exact boundary resolution were integrated into the evaluation script.

5.3 Ablations and Controlled Comparisons

Controlled single-factor ablations provided key insights into model behavior:

- **Bidirectional Conversion Impact (P2 & P3):** Disabling causal attention masks (is_causal = False) on the Modern Decoder increased Phase 2 Strict Macro-F1 from 0.0812 to 0.3735, and Phase 3 Macro-F1 from 0.2714 to 0.3769. Exposing full left-and-right sequence context is vital for sequence tagging and sentence discourse.
- **Adapter Architecture (LORMA vs. LoRA):** On the Hybrid Encoder backbone in Phase 2, LORMA adapters ($r = 128$) achieved 44.9× higher Strict Macro-F1 (0.0981) than standard Hugging Face PEFT LoRA (0.0022). Interleaved rank-minimizing adapter layers captured fine-grained BIO token transitions far more effectively than standard parallel low-rank updates.
- **Deep Weight Adaptation vs. Virtual Prefixes (P7):** In Prior Case Retrieval, adapting internal projections via QLoRA/LORMA achieved 0.1453–0.1542 Micro-F1@7, whereas Prefix Tuning on the Hybrid Encoder collapsed to 0.0118. Virtual prefix tokens failed to warp the frozen backbone's metric space for dense retrieval.

6 Results

Table 4: System performance comparison across all evaluated model variants and phases (including secondary metrics).

Phase	Model Architecture Variant	Adaptation / PEFT Method	Primary Metric	Score	Secondary Metric Scores
P1	Hybrid Prefix-LM (59M)	Pretrained from Scratch	BERTScore F1	0.796455	ROUGE-L: 0.108061
P1	GQA Causal LM (64M)	Pretrained from Scratch	BERTScore F1	0.790354	ROUGE-L: 0.097188
P1	Encoder-Decoder (106M)	Pretrained from Scratch	BERTScore F1	0.783713	ROUGE-L: 0.086844
P1	Modern Decoder (134M)	Pretrained from Scratch	BERTScore F1	0.768136	ROUGE-L: 0.053564
P2	Modern Decoder	LORMA (Rank 128, Bidirectional)	Strict Macro-F1	0.373512	Strict Prec: 0.348576
P2	Hybrid Encoder	LORMA (Rank 128)	Strict Macro-F1	0.098114	Strict Prec: 0.066441
P2	Modern Decoder (Student)	Knowledge Distillation (3-Layer)	Strict Macro-F1	0.082840	Strict Prec: 0.059248
P2	GQA Decoder	LORMA + CRF Head	Strict Macro-F1	0.075512	Strict Prec: 0.054708
P2	GQA Decoder	Houlsby Bottleneck Adapters	Strict Macro-F1	0.074256	Strict Prec: 0.055235
P2	Hybrid Encoder	Standard LoRA (Rank 32)	Strict Macro-F1	0.002186	Strict Prec: 0.001187
P2	Encoder-Decoder	LORMA (Rank 128)	Strict Macro-F1	0.000000	Strict Prec: 0.000000
P3	Modern Decoder	LORMA (Rank 128, Mean Pooling)	Macro-F1	0.376948	Macro-Prec: 0.395100
P3	Encoder-Decoder	LORMA (Rank 128)	Macro-F1	0.313476	Macro-Prec: 0.356945
P3	Hybrid Encoder	LORMA (Rank 16)	Macro-F1	0.297533	Macro-Prec: 0.329027
P3	Modern Decoder (Student)	Knowledge Distillation (10.5M)	Macro-F1	0.295303	Macro-Prec: 0.308338
P3	GQA Decoder	Prefix Tuning (32 Tokens)	Macro-F1	0.271367	Macro-Prec: 0.299411
P3	GQA Decoder	Standard LoRA (Rank 16)	Macro-F1	0.264877	Macro-Prec: 0.282582
P3	GQA Decoder	LORMA (Rank 128)	Macro-F1	0.264048	Macro-Prec: 0.307186
P4	Modern Decoder	LORMA (Rank 16, 1024 Tokens)	Macro-F1	0.544039	Macro-Prec: 0.544209
P4	Modern Decoder (Student)	Knowledge Distillation (2-Layer)	Macro-F1	0.530724	Macro-Prec: 0.538674
P4	Encoder-Decoder	LORMA (Rank 128)	Macro-F1	0.508757	Macro-Prec: 0.613293
P4	GQA Decoder	Standard LoRA (Rank 16)	Macro-F1	0.494154	Macro-Prec: 0.585890
P4	GQA Decoder	BitFit (Bias-Only, 53K Params)	Macro-F1	0.469103	Macro-Prec: 0.539936
P4	GQA Decoder	LORMA (Rank 16, Dual Chunk)	Macro-F1	0.468743	Macro-Prec: 0.515320
P4	Hybrid Encoder	LORMA (Rank 16)	Macro-F1	0.342945	Macro-Prec: 0.727273
P5	GQA Decoder	LORMA (Rank 16, Threshold $\tau = 0.10$)	Macro-F1	0.177300	Micro-F1: 0.318500
P5	Encoder-Decoder	Soft Prompt Tuning (16 Tokens)	Macro-F1	0.100726	Micro-F1: 0.235711
P5	Modern Decoder	LORMA (Rank 16)	Macro-F1	0.062053	Micro-F1: 0.242291
P5	Hybrid Encoder	4-bit NF4 QLoRA (Rank 16)	Macro-F1	0.047063	Micro-F1: 0.251763
P6	Modern Decoder (Student)	Knowledge Distillation (3-Layer)	BERTScore F1	0.821925	ROUGE-1: 0.333589
P6	Modern Decoder (Teacher)	LORMA (Rank 16, $\alpha = 32$)	BERTScore F1	0.814779	ROUGE-1: 0.318791
P6	GQA Decoder	LORMA (Rank 16, $\alpha = 32$)	BERTScore F1	0.793868	ROUGE-1: 0.284619
P6	Encoder-Decoder	QLoRA (Rank 16, $\alpha = 32$)	BERTScore F1	0.782626	ROUGE-1: 0.292672
P6	Hybrid Encoder Causal LM	LoRA (Rank 16, $\alpha = 32$)	BERTScore F1	0.712070	ROUGE-1: 0.003143
P7	Modern Decoder (Teacher)	LORMA (Rank 16, InfoNCE $\tau = 0.05$)	Micro-F1@7	0.154164	Micro-Prec@7: 0.157324
P7	Encoder-Decoder	QLoRA (Rank 16, InfoNCE $\tau = 0.05$)	Micro-F1@7	0.145304	Micro-Prec@7: 0.148282
P7	Modern Decoder (Student)	Knowledge Distillation (InfoNCE + MSE)	Micro-F1@7	0.144123	Micro-Prec@7: 0.147077
P7	GQA Decoder	LORMA (Rank 16, InfoNCE $\tau = 0.05$)	Micro-F1@7	0.135854	Micro-Prec@7: 0.138638
P7	Hybrid Encoder	Prefix Tuning (16 Tokens, Mid-Dim 512)	Micro-F1@7	0.011813	Micro-Prec@7: 0.012055

7 Error Analysis and Discussion

7.1 Phase-Wise Failure Modes

- **P2 Offset Instability & Seq2Seq Collapse:** Fine-tuning the seq2seq Encoder-Decoder under token classification failed to minimize cross-entropy loss (stuck at $\sim$17.48), resulting in zero predictions. Token classification requires direct, non-autoregressive encoder hidden states.
- **P4 Majority-Class Bias in Hybrid Encoder:** Unweighted cross-entropy on the Hybrid Encoder yielded 0.7273 precision but near-zero recall (0.0105), as the model defaulted heavily to predicting the majority outcome class.
- **P5 Logit Suppression under BCE:** Training with unweighted BCE across 100 statute targets pushed average output logits below -4.0. Without post-hoc threshold lowering ($\tau = 0.10$), models predicted zero active statutes for almost all inputs.
- **P6 N-Gram Collapse in Autoregressive Baselines:** While the Hybrid Encoder Causal LM maintained moderate semantic overlap (BERTScore 0.7121), its ROUGE scores collapsed to $\sim$0.0031 due to phrase repetition loops and early EOS emission. Sequence-to-sequence cross-attention and min-length constraints were essential to prevent generation collapse.
- **P7 Prefix Tuning Optimization Breakdown:** Virtual prefix tuning on the frozen Hybrid Encoder exhibited severe optimization instability under InfoNCE loss, peaking at a dev Micro-F1@7 of only 0.0200. Learning rates high enough to steer soft prefixes (1×10^{-2}) caused gradient oscillation.

7.2 What the Metrics Do Not Measure

Standard NLP metrics frequently diverge from actual legal requirements. In Phase 6 (Summarization), ROUGE-L and BERTScore measure n-gram and semantic similarity against a single reference summary. However, a generated summary can achieve a high BERTScore while hallucinating a critical legal procedural detail (e.g., swapping "appellant" for "respondent"). In Phase 2 (NER), Strict Macro-F1 penalizes a model equally whether it completely misidentifies an entity type or misses a single trailing space in a 10-token statute title. In real law firm deployments, high entity recall with slight boundary misalignment is far more valuable than brittle exact boundary matches.

7.3 Limitations and What You Would Do Next

Primary project constraints included limited context windows (1024–2048 tokens) and the absence of external retrieval augmented generation (RAG) databases. Future work should explore: (1) Integrating Ring-Attention or FlashAttention-2 to extend context windows to 32,000 tokens for full legal transcript encoding; (2) Pretraining with explicit structural objectives (e.g., masked legal citation prediction); (3) Implementing direct preference optimization (DPO) using synthetic legal preference pairs to align abstractive summarization safety.

7.4 Surprising Findings

Distilled Student Outperforms Teacher in Summarization: In Phase 6, the 3-layer Modern Decoder student trained via logit-level distillation achieved a higher BERTScore F1 (0.8219) than its 6-layer teacher (0.8147). Soft-target KL-divergence acted as a powerful regularizer, smoothing over-confident teacher distributions and preventing hallucination loops.

8 Conclusion

This project demonstrated that compact, domain-specific legal language models (50M–134M parameters) pretrained from scratch can achieve competitive performance across complex downstream legal NLP tasks without relying on massive external LLMs. Three core technical findings emerged:

1. **Hybrid Prefix Pretraining and Bidirectional Fine-Tuning:** Pretraining with hybrid prefix attention (BERTScore 0.7965) and converting causal decoders to full bidirectional attention for downstream tagging (+10.55 F1 gain in P3) provides critical context conditioning for legal texts.

2. **Adapter Superiority Over Prefix Tuning:** Deep low-rank matrix adaptation (LORMA and QLoRA) consistently outperformed virtual prefix tuning, providing stable contrastive alignment and fine-grained token classification.

3. **Regularization via Distillation:** Logit-level knowledge distillation effectively compresses model footprints by 25–75% while maintaining or improving task accuracy.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: Gemini.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explaining LORMA, GQA, InfoNCE contrastive dynamics, and PEFT methods.
Ideation / brainstorming at a high level	Yes	Brainstorming dual-anchor chunking and bidirectional attention conversions.
Debugging small ancillary snippets	Yes	Fixing offset mapping alignment in P2 and logit suppression threshold search in P5.
Plotting / visualization code	Yes	Generating performance tables and result comparison formatting.
Grammar / writing style of the report	Yes	Polishing technical prose and ensuring academic tone throughout the report.
Report structure / section wording	Yes	Formatting sections and typesetting mathematical formulations into clean HTML.
Designing the model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	Yes	Assisting in custom BPE tokenizer scripts and InfoNCE loss implementation.
Repository documentation	Yes	Writing modular README files for GitHub submission tracking.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. I referenced the official IL-TUR legal benchmark paper [1], PyTorch documentation, Hugging Face Tokenizers library (used exclusively for custom BPE training), PEFT library implementations, and evaluation packages ("bert-score", "rouge-score", "scikit-learn"). No pretrained language model weights were used for any backbone; 'roberta-large' was used strictly as an evaluation metric calculator.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

Answer: I designed the multi-architecture pretraining matrix, selected model hyperparameters, and configured custom BPE tokenizers. I personally executed all training pipelines on dual NVIDIA Tesla T4 GPUs, monitored loss convergence, handled CodaBench submissions, and resolved critical failure modes (e.g., offset mapping alignment for P2 L-NER, decision threshold search for P5 LSI, and knowledge distillation setups for P6 summarization). Two notable failed attempts provided key insights: (1) standard LoRA on the Hybrid Encoder failed completely in P2 (Strict F1: 0.0022) compared to LORMA (0.0981), showing that rank-minimizing interleaved layers are crucial for token-level BIO boundaries; and (2) Prefix Tuning in P7 collapsed (Micro-F1@7: 0.0118) compared to QLoRA (0.1453), proving that soft virtual prefixes cannot warp a frozen representation space

for contrastive bi-encoder retrieval.

References

[1] Joshi, A., Paul, S., Sharma, A., Goyal, P., Ghosh, S., & Modi, A. (2024). *IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning*. Proceedings of ACL 2024.

[2] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). *BERTScore: Evaluating Text Generation with BERT*. ICLR 2020.

[3] Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. Text Summarization Branches Out, 74-81.

[4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.

[5] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023.

[6] Bihany, B., Patel, A., & Modi, A. (2025). *LoRMA: Low-Rank Multiplicative Adaptation for LLMs*. Findings of ACL 2025.

[7] Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. ACL 2021.

[8] Ben Zaken, E., Goldberg, Y., & Ravfogel, S. (2022). *BitFit: Simple Parameter-Efficient Fine-Tuning for Transformer-Based Masked Language-Models*. ACL 2022.

[9] Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531.

[10] Ainslie, J., et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP 2023.

[11] Raffel, C., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR, 21(140), 1-67.

[12] Exploration-Lab. *CS-781-Capstone official dataset repository*.